Uso de la API de Producteca

producteca

Uso de la API de Producteca 1
¿Qué necesito para conectarme? 2
Recursos 2
Manejo de respuestas 2
Productos 3
Búsqueda de productos 3
Ejemplo con paginado, canal de venta y marca 4
Creación de productos 4
Creación de variantes 6
Ejemplo de creación de productos con un atributo de variación 7
Sincronización de precios 8
Sincronización de stocks 9
Sincronización de precios y stocks 11
Sincronización de stock de seguridad por canal 11
Sincronización de ofertas 12
Sincronización de impuestos (solamente para Argentina) 13
Ventas 15
Busqueda de ventas 15
Ejemplo con paginado, estado de venta, estados de envío y canal 16
Guia de uso para levantar ventas 16
Busqueda de ventas con filtro de estado y fecha 17
Busqueda de ventas individuales 17
Marcar venta como sincronizada y dejar de reservar stock 18
Busqueda de ventas sin sincronizar con pago aprobado 18
Busqueda de ventas por id de integración 19
Marcar la venta como sincronizada y adjuntar la factura 19
Adjuntar la factura en una venta 20
Busqueda de ventas canceladas 21
Obtener la etiqueta de una venta 21
Creación de envíos 22
Actualización de envíos 23
Notificaciones mediante Webhooks 24
Anexo 1: Esquema resumido de una venta en Producteca 26
Anexo 2: Código de "salesChannel" por canal 30

¿Qué necesito para conectarme?

- Un sistema propio que pueda realizar llamadas a una API.
- Credenciales: son claves fijas y se asignan directamente a cada cliente. Las mismas serán facilitadas por Producteca durante el inicio del proyecto
    - API Key: es tu llave de acceso única.
    - Bearer Token: un código de seguridad que confirma tu identidad.

Recursos

Pueden acceder a la descripción detallada de todos los endpoints, parámetros y esquemas de respuesta en Swagger Hub.

Esta documentación resume los puntos más relevantes del uso de la API, pero no cubre la totalidad de los casos ni parámetros disponibles, recomendamos consultar también el Swagger Hub para obtener el detalle completo y actualizado.

Manejo de respuestas

En las consultas de de sincronización, las operaciones son sincrónicas, por lo que el código de estado de la respuesta HTTP es el principal indicador a considerar:

2xx – Éxito: la operación se realizó correctamente.

El código específico puede variar según si la solicitud generó o no cambios sobre los datos.

4xx – Error del cliente: se produjo un error por falta de autorización, error de credenciales, sintaxis incorrecta, datos inválidos o problemas de permisos.

En estos casos, el cuerpo de la respuesta suele incluir detalles adicionales sobre la causa del error.

5xx – Error del servidor: indica fallas de conexión o problemas internos en el servicio.

Adicionalmente, existen restricciones en la cantidad de solicitudes por segundo según el tipo de entorno. En el entorno productivo se admite un límite sostenido de 100 consultas por segundo.

Productos

Antes que nada, es importante entender que Producteca devuelve los resultados de cualquier búsqueda de search, de manera paginada.
Para poder recorrer la totalidad de dichos resultados, es necesario utilizar los parámetros de top y skip:

```javascript
top = 100 //devuelve 100 resultados por página
skip = 200 //muestra a partir del resultado 201
```

Búsqueda de productos

```javascript
curl --location --request GET
'https://api-external.producteca.com/search/products?$filter=&isArchived=fal
se&top=&skip=&salesChannel=2' \
--header 'Content-Type: application/json' \
--header 'X-api-key:  {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'
```

Los filtros se arman como un único parámetro:

$filter=(condicion1)and(condicion2)...and(condicionN)

**Todas las requests de `search/products` deben tener el parámetro `salesChannel=2` para los productos funcionales `(isArchived=false)`, para los productos archivados `(isArchived=true)` no es necesario enviar el parámetro.*

Algunos ejemplos son:

Filtro	**Condiciones**	**Comentarios**
Sku	`(skus/any(x: x eq '1234'))`	
Id de Producteca	`(productId eq 123456789)`	
Canal de venta	`(integrationApps/any(x1: x1 eq '490'))`	Disponible en el Anexo 2
Estado de la publicación por canal	`(status eq 'Active')`	Opciones: Active, Paused, __empty_value__ (sin publicar)
Según marca	`(brand eq 'Adidas')`	
Según tag principal	`(category eq 'Calzado')`	

Según id de venta del canal	`(integrationIds/any(x: x eq '123456'))`	
Según estado del producto: Sin publicar	`(isPublished eq false)`	

Van a obtener una respuesta con este formato:

```javascript
{
    "count": 155,
    "results": [
      {...
```

El `"count"` indica la cantidad total de resultados, deben recorrer todas las páginas hasta que el array de `"results"` esté vacía: `"results": []`

Ejemplo con paginado, canal de venta y marca

```javascript
curl --location --request GET
'https://api-external.producteca.com/search/products?skip=0&top=10&$filter=(
integrationApps/any(x1: x1 eq '490')) and (brand eq 'Adidas')' \
--header 'Content-Type: application/json' \
--header 'X-api-key:  {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'
```

En este caso, trae los primeros 10 productos, publicados en el canal Macro Premia (490) con marca “Adidas”.

Creación de productos

Para la creación de productos en Producteca, se debe ejecutar una consulta por cada producto, con este formato:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize'\
--header 'Content-Type: application/json' \
--header 'X-api-key:  {{apiKey}}' \
--header 'Authorization:  Bearer {{token}}' \
--header 'createifitdoesntexist: true' \
--data-raw '{
    "sku": "104003010BA",
    "code": "104003010",
```

```
    "name": "Televisor",
    "attributes": [{
        "key": "Pulgadas",
        "value": "50"
    }],
    "tags": {
        "append": ["Televisor", "Prueba BNA", ],
        "remove": ["prueba2"]
    },
    "brand": "Sony",
    "notes": "Descripción larga en texto plano",
    "dimensions": {
        "weight": 5000, //Peso medido en gramos (gr)
        "width": 17, //Ancho medido en centimeros (cm)
        "length": 93, //Largo medido en centimeros (cm)
        "height": 65, //Altura medido en centimeros (cm)
        "pieces": 1
    },
    "stocks": [{ "quantity": 200, "warehouse": "Default" }],
    "prices": [{ "amount": 1423, "priceList": "Default" }],
    "pictures": [{ "url": "urlFoto1" }, { "url": "urlFoto2" }],
    "$updatableProperties": ["stocks", "prices"]
}'
```

!! Es muy importante mencionar que el campo `"sku"` es de carácter **OBLIGATORIO**.

El header `createifitdoesntexist` debe estar y ser `true` para que los productos se creen y ser `false` si no creemos que se creen nuevos productos.

El campo `"code"` identifica al **producto**, mientras que el campo `"sku"` identifica a cada **variante** dentro de ese producto.
Si el catálogo que se desea crear va a tener una única variante en cada publicación, no es necesario enviar el campo `"code"`. Por el contrario, si el catálogo que se desea crear tendrá más de una variante, en ese caso es obligatorio enviar el campo `"code"`.
Tener en cuenta que si se está creando un producto con variantes, tanto el campo `"code"` como todos los campos que sean a nivel producto, deben ser iguales entre las distintas variantes.

El campo `"tags"` opera como lista, y permite filtrar los productos en base a la información que envían en cada elemento de esta lista. En general, se suele usar el tipo de producto como tag base, pero luego se pueden agregar (`append`) o quitar (`remove`) cualquiera de estos datos. No es un campo obligatorio, simplemente puede llegar a facilitar la gestión del catálogo en Producteca para el equipo operativo, ya que se pueden utilizar para filtrar en la interfaz.

El campo `$updatableProperties` se utiliza para indicar qué **campos** pueden **actualizarse** luego de la creación del producto. Todo lo que no esté listado dentro del array se utiliza solo en la creación inicial, y no se vuelve a pisar en actualizaciones posteriores. De esta manera se evita sobrescribir información de productos que ya están publicados en los canales de venta.
En el caso de no indicarlo, todos los campos son actualizables, y existe el riesgo de sobrescribir la información del producto, por lo que es necesario que esté bien definido.

Creación de variantes

Para crear variantes en Producteca (por talle, color o ambas), primero se debe crear el **producto base** con toda la información común entre todas las variantes: código, marca, atributos generales, EAN, dimensiones; y la información particular correspondiente a la **primera variante**: SKU, **atributos de variación** (talle, color o ambos).
Es importante recordar que el campo `"sku"` identifica a cada **variante** dentro del producto, y el campo `"code"` identifica al **producto**, por lo que debe coincidir entre todas las variantes, ya que es el campo que Producteca utiliza para agruparlas.

Para crear las variantes, se debe ejecutar una consulta individual por cada variante, utilizando el siguiente formato:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}' \
--header 'createifitdoesntexist: true' \
--data '{
    "code": "VESTIDO", //comun entre todas las variantes
    "sku": "VESTIDOROSA",
    "variationAttributes": [ { "key": "Color", "value": "Rosa" },
        { "key": "Talle", "value": "12" } ]
}'
```

Ejemplo de creación de productos con un atributo de variación

Para crear un producto con talle y color, agrupados por color en Producteca, es decir un único atributo de variación que sea el talle, se debería crear primero el producto con la información a nivel producto y la primer variante:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize'\
```

```
--header 'Content-Type: application/json' \
--header 'X-api-key:  {{apiKey}}' \
--header 'Authorization:  Bearer {{token}}' \
--header 'createifitdoesntexist: true' \
--data-raw '{
    "sku": "VESTIDOROJO10",
    "code": "VESTIDOROJO",
    "name": "Vestido largo",
    "attributes": [{ "key": "Coleccion", "value": "Verano" }],
    "tags": {
        "append": ["Vestido", "Prueba Mercado Libre", ],
        "remove": ["prueba2"]
    },
    "brand": "Zara",
    "notes": "Descripción larga en texto plano",
    "dimensions":{ "weight": 5000, "width": 17, "length": 93, "height": 65},
    "variationAttributes": [
        { "key": "Color", "value": "Rojo" },
        { "key": "Talle", "value": "10" }
    ],
    "stocks": [{ "quantity": 200, "warehouse": "Default" }],
    "prices": [{ "amount": 1423, "priceList": "Default" }],
    "pictures": [{ "url": "urlFoto1" }, { "url": "urlFoto2" }],
    "$updatableProperties": ["stocks", "prices"]
}'
```

Y luego para crear las variantes, se debe ejecutar una consulta individual por cada variante, utilizando el siguiente formato:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}' \
--header 'createifitdoesntexist: true' \
--data '{
    "code": "VESTIDOROJO",
    "sku": "VESTIDOROJO12",
    "variationAttributes": [
        { "key": "Color", "value": "Rojo" },
        { "key": "Talle", "value": "12" }
    ]
}'
```

Es importante que el campo `"code"` coincida entre todas las variantes, ya que es el campo que Producteca utiliza para agruparlas.

Sincronización de precios

Para sincronizar precios la consulta a ejecutar es la siguiente:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'\
--data-raw '{
    "sku": "104003010BA",
    "prices": [{ "priceList": "Lista 1", "amount": 2999 },
        { "priceList": "Lista 2", "amount": 5999 }
    ]
}'
```

Se debe realizar un llamado por cada producto, no es posible actualizar múltiples productos en una misma consulta.

En una misma consulta se pueden cargar distintos precios, en múltiples listas de precios[1] en Producteca, para esto las listas deben estar creadas previamente en su cuenta.
En el campo `"priceList"` debe colocar el nombre de la lista de precios a actualizar, y en el campo `"amount"` se debe colocar el valor numérico del precio para esa lista. Se puede ver en el ejemplo anterior que existen 2 listas de precios, una llamada "Lista 1" y otra "Lista 2".

De ejecutarse correctamente, obtendrán la siguiente respuesta:

```javascript
[
    {
        "product": {
            "updated": true
        }
    }
]
```

Sincronización de stocks

Antes de revisar las consultas, es importante entender los diferentes tipos de stock dentro de Producteca, la siguiente fórmula y sus datos de entrada:

$$Disponible\ (u) = Stock\ (u) - Reservado\ (u) \qquad (4.1)$$ [2]

---

[1] *Para aprender a crear las listas de precios, pueden acceder al siguiente enlace.*
[2] *Para poder entender más sobre los distintos tipos de stock, pueden acceder al siguiente enlace.*

- El **stock** (u) dentro de Producteca es la cantidad **física** total de unidades que se tiene de un producto, incluye todas las unidades que están en tu depósito, sin descontar las unidades reservadas para ventas. Esta es la cantidad de unidades de stock informado hacia Producteca para un producto.
- El **reservado** (u) es la cantidad de unidades que ya están **reservadas para pedidos** que ingresaron desde algún canal de ventas, pero que todavía no fueron despachados o cancelados. Estos productos siguen físicamente en tu depósito, pero no deberían venderse nuevamente para evitar sobreventa.
- El **disponible** (u) es la cantidad de unidades que realmente podés vender, una vez descontadas las unidades que ya están reservadas para pedidos; esta cantidad es la que Producteca informa a los canales de venta. Es la **diferencia** entre el stock físico y el reservado. De esta manera solamente se disponibiliza el stock aún no se encuentra reservado en los canales de venta.

Ejemplo: Si informan en un producto **10** unidades de **stock**, y tiene **3** unidades **reservadas** de ventas que ingresaron, el **disponible** que se envía a los canales de venta va a ser de **7** unidades (10 stock - 3 reservadas).
Si luego informan a Producteca que el **stock** es de **20** unidades y no dejan de reservar el stock, el reservado se mantiene en 3 unidades, y ahora el disponible es de 17 unidades.
Cuando se levanten las ventas y se deje de reservar el stock de las mismas (las 3 unidades), el **disponible** será **17 unidades**, el **stock** será **17 unidades** y el **reservado** será **0 unidades**.
*Para disminuir el stock reservado, se puede dar porque el envío de la venta pasa a estado `"InTransit"` o `"Done"`, o porque envían la baja del reservado vía API[3].

La consulta para sincronizar stocks es la siguiente:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'\
--data-raw '{
    "sku": "12345678",
    "stocks": [{
        "warehouse": "Default",
        "quantity": 10
    }]
}'
```

[3] *Para saber como dejar de reservar stock por API, se puede ver la consulta en este mismo documento, en el apartado de "Guia de Uso para levantar ventas"*

Para poder impactar el stock en un depósito en Producteca, es necesario asignar el nombre del depósito creado previamente en Producteca[4].
Se debe realizar un llamado por cada producto, no es posible actualizar múltiples productos en una misma consulta.
En el campo `"warehouse"` se debe colocar el nombre del depósito de Producteca en el cual desean informar el stock. En el ejemplo el nombre del depósito se llama `"Default"`.
En el campo `"quantity"` se debe informar la cantidad de stock que se desea informar a Producteca en dicho depósito. Dicha cantidad es el *Stock (u)* de la fórmula anterior.

De ejecutarse correctamente, obtendrán la siguiente respuesta:

```javascript
[
    {
        "variation": {
            "updated": true
        }
    }
]
```

Sincronización de precios y stocks

Para poder sincronizar precios y stocks en el mismo llamado, la consulta que deben ejecutar es la siguiente:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'\
       --data-raw '{
           "sku": "104003010BA",
           "prices": [ { "priceList": "Lista 1", "amount": 2999 } ],
           "stocks": [ { "warehouse": "Default", "quantity": 10 } ]
       }'
```

De ejecutarse correctamente, obtendrán la siguiente respuesta:

```javascript
[
    {
        "product": {
```

[4] *Para aprender a crear o eliminar depósitos, se puede ingresar al siguiente enlace.*

```
            "updated": true
        }
    },
    {
        "variation": {
            "updated": true
        }
    }
]
```

Sincronización de stock de seguridad por canal

El stock de seguridad es el nivel mínimo de unidades que podés configurar para tus productos publicados en los diferentes marketplaces, sirve para **evitar ventas** de productos **sin stock**[5]: al llegar a ese número, las publicaciones se **pausarán** automáticamente en caso de tener la sincronización de stocks con el canal encendida.
El stock de seguridad se define por producto y por marketplace, para poder enviar stock de seguridad es necesario que el producto ya cuente con la integración al canal, es decir publicado y vinculado a Producteca.
En el body de la consulta, se debe colocar el número de `"app"` con el valor correspondiente al marketplace a configurar el stock de seguridad para dicho sku. El listado de los números de apps para cada canal de venta lo pueden encontrar en el Anexo 2.
En el campo `"safetyStock"` se coloca la cantidad de unidades a configurar para dicho SKU en dicho número de app.

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'\
--data-raw '{
    "sku": "12345678",
    "integration": {
        "app": 2,
        "safetyStock": 96
    }
}'
```

[5] *Para poder entender más sobre las distintas entidades del stock, pueden acceder al siguiente enlace.*

Sincronización de ofertas

Para poder realizar esta sincronización, es necesario primero contar con una campaña de descuento creada en Producteca[6].
Una vez que está creada la campaña, pueden ver el correspondiente ID unívoco, para colocarlo en el campo `"campaign"` de la consulta. Asimismo, para poder ejecutar la consulta correctamente, es importante que el valor de `"dealPrice"` sea **estrictamente menor** que el valor de `"regularPrice"`.
Para poder sincronizar ofertas, la consulta que deben ejecutar es la siguiente:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'\
--data-raw '{
    "sku": "104003010BA",
    "deals": [
        {
            "campaign": "12345678",
            "regularPrice": 2999,
            "dealPrice": 1999
        }
    ]
}'
```

Para quitar la oferta del producto, se puede enviar la misma consulta colocando el mismo valor en `"regularPrice"` y en `"dealPrice"`.
Comentario: Esta funcionalidad no se puede validar en un ambiente de pruebas, únicamente en producción.
Aclaración: No todos los canales permiten la actualización de ofertas vía API. Es importante consultar previamente para entender si los canales a integrar permiten este desarrollo.

Sincronización de impuestos (solamente para Argentina)

Para la sincronización de impuestos en Argentina, deben ejecutar la siguiente consulta:

```javascript
curl --location --request POST
'https://api-external.producteca.com/products/synchronize' \
--header 'Content-Type: application/json' \
```

[6] *Para aprender a crear campañas de descuento desde Producteca, se puede acceder al siguiente enlace.*

```
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'\
--data-raw '{
    "sku": "12345678",
      "taxes": [{ "name": "IVA", "value": "21" },
        { "name": "IMP_INT", "value": "25" }]
}'
```

Los impuestos a cargar deben ser la tasa **efectiva** del **IVA** (21, 10.5, 27) y la tasa **nominal** de los **Impuestos Internos** (25, 10, etc).

Para realizar la validación de la carga, pueden ejecutar la siguiente consulta:

```javascript
curl --location --request GET
'https://api-external.producteca.com/taxes?productId={productId}' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'\
```

Ventas

Antes que nada, es importante entender que Producteca devuelve los resultados de cualquier búsqueda de search, de manera paginada.
Para poder recorrer la totalidad de dichos resultados, es necesario utilizar los parámetros de top y skip:

```javascript
top = 100 //devuelve 100 resultados por página
skip = 200 //muestra a partir del resultado 201
```

Busqueda de ventas

```javascript
curl --location --request GET
'https://api-external.producteca.com/search/salesorders?$filter=&top=&skip='
\
--header 'Content-Type: application/json' \
--header 'X-api-key:  123456' \
--header 'Authorization: Bearer {{token}}'
```

Los filtros se arman como un único parámetro:

$filter=(condicion1)and(condicion2)...and(condicionN)

Algunos ejemplos son:

Filtro	**Condiciones**	**Comentarios**
Estado de la venta	`(status eq 'Opened')`	Opciones: Opened, Closed, Cancelled
Canal de venta	`((salesChannel eq 2) or (salesChannel eq 220)or (salesChannel eq 59))`	Disponible en el Anexo 2
Fecha (rango)	`((date gt 2025-11-03T03:00:00Z) and (date lt 2025-11-08T02:59:59Z ))`	`gt` es la cota inferior, y `lt` es la cota superior
Estado del pago	`(paymentStatus eq 'Approved')`	Opciones: Pending, OnDelivery, InProcess, InMediation, Approved, Rejected, Cancelled, Refunded, ChargedBack

Estado de la entrega	`(deliveryStatus eq 'Done')`	Opciones: ToBeConfirmed, PickingPending, ReadyToShip, ToBePickedUp, InTransit, Done, ToBeReturned, InReturn, Returned
Según si tiene envío	`(hasShipments eq true)`	
Según id de venta del canal	`(integrationIds/any(x: x eq '2000012074281140'))`	
Sin sincronizar	`(invoiceIntegrationId eq null)`	

Van a obtener una respuesta con este formato:

```javascript
{
    "count": 155,
    "results": [
      {...
```

El `"count"` indica la cantidad total de resultados, deben recorrer todas las páginas hasta que el array de `"results"` esté vacía: `"results": []`

Ejemplo con paginado, estado de venta, estados de envío y canal

```javascript
curl --location --request GET
'https://api-external.producteca.com/search/salesorders?skip=0&top=10&$filte
r=(status eq 'Opened') and (deliveryStatus eq 'PickingPending' or
deliveryStatus eq 'ReadyToShip') and (salesChannel eq 2)' \
--header 'Content-Type: application/json' \
--header 'X-api-key:  123456' \
--header 'Authorization: Bearer {{token}}'
```

En este caso, trae las primeras 10 ventas, con estado “Abierta”, Estados de entrega “Pendiente” o “Listo para enviar”, del canal Mercado Libre.

Guia de uso para levantar ventas

En primer lugar, deben levantar ventas con filtro de estado y fecha

Busqueda de ventas con filtro de estado y fecha

```javascript
curl --location --request GET
'https://api-external.producteca.com/search/salesorders?$filter=(date gt
2025-11-03T03:00:00Z) and (date lt 2025-11-08T02:59:59Z) and (status eq
'Opened')' \
--header 'Content-Type: application/json' \
--header 'X-api-key:  123456' \
--header 'Authorization: Bearer {{token}}'
```

→ en este caso se buscan ventas desde el `2025-11-03` (`gt`) hasta el `2025-11-07` (`lt`) (se debe colocar `2025-11-08T02:59:59Z`)

Van a obtener una respuesta con este formato:

```javascript
{
    "count": 2,
    "results": [
       {...},
       {...}
    ]
}
```

Luego, con el campo `"orderId"` de cada objeto dentro de `"results"` de la respuesta anterior, realizan la búsqueda de la venta particular para obtener la información completa de la misma.

Busqueda de ventas individuales

```javascript
curl --location --request GET
'https://api-external.producteca.com/salesorders/{orderId}' \
--header 'Content-Type: application/json' \
--header 'X-api-key:  123456' \
--header 'Authorization: Bearer  {{token}}'
```

Luego, deben marcar la venta como sincronizada para dejar de reservar stock.

Marcar venta como sincronizada y dejar de reservar stock

Para poder indicar que han levantado correctamente cada venta, deben marcarla como sincronizada para que quede reflejado en Producteca, y enviar el campo decreaseStock para que la venta deje de reservar unidades de stock.

```javascript
curl --location --request POST
'https://api-external.producteca.com/salesorders/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}' \
--data-raw '{
    "id": {orderId},
    "invoiceIntegration": {
        "app": 249,
        "integrationId": "ERP123456789",
        "decreaseStock": true
    }
}'
```

El campo `"integrationId"` debería ser el ID de comprobante en su sistema, para que luego lo puedan identificar en Producteca.
El parámetro `"app": 249` sirve para denotar que la venta fue sincronizada mediante el uso de la API externa.
!! Asimismo, es muy importante que una vez que levanten la venta envíen el parámetro `"decreaseStock": true` para que Producteca deje de comprometer esas unidades de stock que está informando al canal (4.1).
Al momento de colocar este campo en true se considera que su sistema ya contempla la baja de esas unidades de stock al informar el stock de ese producto. Hasta ese momento Producteca continúa reservando el stock para prevenir el quiebre de stock.

Una vez que comiencen a marcar las ventas como sincronizadas, pueden realizar una búsqueda de ventas sin sincronizar.

Busqueda de ventas sin sincronizar con pago aprobado

```javascript
curl --location --request GET
'https://api-external.producteca.com/search/salesorders?$filter=status eq
'Opened' and paymentStatus eq 'Approved' and invoiceIntegrationApp eq null'
\
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'
```

En el caso de que una venta tenga múltiples pagos, figurará en los resultados solo si tiene todos los pagos aprobados.

Busqueda de ventas por id de integración

Para realizar la búsqueda de una venta según el valor de id de integración (`integrationId`) que indicaron cuando la marcaron como sincronizada, la consulta es la siguiente:

```
JavaScript
curl --location --request GET
'https://api-external.producteca.com/search/salesorders?$filter=integrationI
ds/any(x: x eq 'ERP123456789')' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'
```

Marcar la venta como sincronizada y adjuntar la factura

Para marcar la venta como sincronizada y adjuntar la factura en el mismo movimiento, la consulta es la siguiente:

```
JavaScript
curl --location --request POST
'https://api-external.producteca.com/salesorders/synchronize' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}' \
--data-raw '{
    "id": {orderId},
    "invoiceIntegration": {
        "app": 249,
        "integrationId": "ERP123456789",
        "decreaseStock": true,
        "documenturl": "urlDelPdf"
    }
}'
```

En el caso de querer adjuntar un .XML, el body de la consulta debería ser el siguiente:

```
JavaScript
--data-raw '{
    "id": {orderId},
    "invoiceIntegration": {
           "app": 249,
           "integrationId": "ERP123456789",
           "decreaseStock": true,
           "xmlurl": "urlDelXml"
       }
```

```
}'
```

Aclaración: El formato admitido de la factura puede variar según el marketplace. Es importante consultarlo previamente para validar el formato del canal a integrar.

Adjuntar la factura en una venta

Es requisito que la venta esté sincronizada –tenga un `"invoiceIntegration"`– para poder adjuntar la factura, ya sea la URL en formato .PDF o .XML, de la misma.

```javascript
curl --location -g --request PUT
'https://api-external.producteca.com/salesorders/{orderId}/invoiceIntegratio
n' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'
--data-raw '{
  "invoiceIntegration": {
    "app": 249,
    "documenturl": "urlDelPdf",
    "integrationId": "{integrationId}"
  }
}'
```

En el caso de querer adjuntar un .XML, el body de la consulta debería ser el siguiente:

```javascript
curl --location -g --request PUT
'https://api-external.producteca.com/salesorders/{orderId}/invoiceIntegratio
n' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'
--data-raw '{
  "invoiceIntegration": {
    "app": 249,
    "xmlurl": "urlDelXml",
    "integrationId": "{integrationId}"
  }
}'
```

Busqueda de ventas canceladas

Para realizar la búsqueda de ventas canceladas, la consulta es la siguiente:

```JavaScript
curl --location --request GET
'https://api-external.producteca.com/search/salesorders?$filter=status eq
'Cancelled'\
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'
```

Aclaración: las ventas canceladas no reservan stock.

Obtener la etiqueta de una venta

Para obtener la etiqueta de una venta, la misma debe estar en Producteca, y la consulta es la siguiente:

```JavaScript
curl --location --request GET
'https://api-external.producteca.com/salesorders/{orderId}/labels' \
--header 'Content-Type: application/json' \
--header 'X-api-key:  123456' \
--header 'Authorization: Bearer  {{token}}' \
```

Producteca solo puede devolver etiquetas o guías de envíos que aún no hayan sido despachados. Una vez que el paquete está en tránsito, ya no es posible volver a solicitar estos documentos, para evitar la generación accidental de una etiqueta duplicada y posibles errores logísticos.
Además, no es posible consultar etiquetas correspondientes a envíos de fulfillment o pick-up, ya que esos flujos son gestionados directamente por las plataformas o proveedores involucrados.

Creación de envíos

Para crear el envío de una venta, la consulta es la siguiente:

```JavaScript
curl --location --request POST
'https://api-external.producteca.com/salesorders/{orderId}/shipments' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}'\
--data-raw '{
    "method": {
```

```
        "trackingNumber": "numeroDeSeguimiento",
        "trackingUrl": "urlDeSeguimiento",
        "status": "ReadyToShip",
        "courier": "FedEx",
    }
}'
```

En el caso de crear más de un envío por venta, será necesario que indiquen los productos y variantes que correspondan en cada envío, en el body de la consulta de esta forma:

```javascript
--data-raw '{
    "products": [{
        "product": 202922451, //id del producto
        "variation": 369273630, //id de la variante
        "quantity": 1
    }],
    "method": {
        "trackingNumber": "numeroDeSeguimiento",
        "trackingUrl": "urlDeSeguimiento",
        "status": "ReadyToShip",
        "courier": "FedEx",
    }
}'
```

Van a obtener una respuesta con este formato:

```javascript
{
    "date": "2025-11-12T21:07:00.518864+00:00", //fecha y hora del momento
    "products": [...], //productos de la venta
    "method": {
        "trackingNumber": "numeroDeSeguimiento",
        "trackingUrl": "urlDeSeguimiento",
        "courier": "FedEx",
        "mode": null,
        "cost": 0,
        "type": "Ship",
        "eta": 0,
        "status": "ReadyToShip"
    },
    "id": 110951151 //id del shipment {shipmentId}
}
```

Actualización de envíos

Para actualizar el status de un envío, con el id que obtuvieron en la respuesta anterior, pueden realizar la siguiente consulta:

```javascript
curl --location --request PUT
'https://api-external.producteca.com/salesorders/{orderId}/shipments/{shipme
ntId}' \
--header 'Content-Type: application/json' \
--header 'X-api-key: {{apiKey}}' \
--header 'Authorization: Bearer {{token}}' \
--data '{
    "method": {
        "status": "InTransit"
    }
}'
```

Siempre se recomienda actualizar los status de los envíos en Producteca para que las ventas no queden abiertas.

Notificaciones mediante Webhooks

Los webhooks son notificaciones que Producteca envía a su sistema cuando ocurre cierto evento, se utilizan cuando quieren escuchar cambios sobre productos o ventas.

Los eventos que pueden disparar webhooks son los siguientes:
Productos:

- Creación
- Publicación
- Stocks: cambio de stock
- Precios: cambio de precio

Ventas:

- Creación
- Estado de pago: cambio en el estado del pago: a aprobado o cancelado según corresponda
- Estado de envío: cambio en el estado de entrega
- Sincronización
- Cancelación

Las notificaciones se envían en bloques de 2 minutos, con un timeout de 30 segundos y se reintenta 10 veces en caso de que el endpoint del cliente falle, hasta 3 días como máximo.

En ambos casos se realiza un POST a la url especificada por el cliente con los siguientes parámetros: `{companyId, resourceType, resourceId }`

- `companyId`: id de la cuenta de Producteca
- `resourceType`: recurso por el que se está enviando el webhook, puede ser `products` o `salesOrders`
- `resourceId`: es el id del recurso por el que se está enviando el webhook, puede ser el id del producto o el id de la venta en Producteca

Ejemplo:

```javascript
curl --location --request POST
'{urlDelSeller}?companyId={companyId}&resourceType=salesOrders&resourceId={s
alesOrderId}'

curl --location --request POST
'{urlDelSeller}?companyId={companyId}&resourceType=products&resourceId={prod
uctId}'
```

Del lado del seller, es necesario implementar un proceso que:

1. Escuche las notificaciones enviadas por Producteca,

2. Identifique el tipo de cambio recibido y la entidad a la que hace referencia (por ejemplo, `companyId + product/{id}` o `salesOrder/{id}`),
3. Busque la entidad correspondiente utilizando el id y obtenga la información actualizada.

    La respuesta incluye todos los datos del producto o venta, no solo los campos que hayan cambiado.

Anexo 1: Esquema resumido de una venta en Producteca

JavaScript

```
{
    "tags": [ "tag1", "tag2" ],
    "integrations": [
        {
            "alternateId": null,
            "integrationId": "1793258127", //ID de la venta del canal
            "app": 345 //nro de canal de venta (ver Anexo2)
        }
    ],
    "invoiceIntegration": {...}, //id de integración
    "channel": "345", //canal de venta
    "piiExpired": false,
    "contact": { //datos de contacto
        "name": null,
        "contactPerson": "Pedro Producteca",
        "mail": "pedroproducteca@hotmail.com",
        "phoneNumber": "+541198761234",
        "taxId": " ",
        "location": {
            "streetName": "Pueyrredon",
            "streetNumber": "80",
            "addressNotes": "La casa con rejas verdes",
            "state": "Buenos Aires",
            "city": "Pergamino",
            "neighborhood": null,
            "zipCode": "2700"
        },
        "notes": null,
        "type": "Customer",
        "priceList": null,
        "priceListId": null,
        "profile": null,
        "billingInfo": { //datos de facturación
            "docType": "DNI",
            "docNumber": "12345678",
            "streetName": "Pueyrredon",
            "streetNumber": "80",
            "comment": "",
            "zipCode": "2700",
            "neighborhood": null,
            "city": "Pergamino",
            "state": "Buenos Aires",
            "businessName": "",
            "stateRegistration": "",
            "taxPayerType": "Consumidor Final",
```

```
            "firstName": "Pedro",
            "lastName": "Producteca",
            "economicActivity": null
        },
        "hasLocationReady": true,
        "id": 123456789
    },
    "lines": [ //productos de la venta
        {
            "price": 100, //precio
            "originalPrice": null,
            "transactionFee": null,
            "product": {
                "name": "Producto1", //nombre
                "code": "ProductosA", //codigo agrupador
                "brand": null,
                "id": 987654321 //id del producto en Producteca
            },
            "variation": {...},
            "orderVariationIntegrationId": null,
            "quantity": 1, //cantidad de este item en la venta
            "conversation": {...},
            "reserved": 0,
            "id": 181455541
        }
    ],
    "warehouse": "Default", //depósito del que se reserva el stock
    "warehouseId": 577832,
    "warehouseIntegration": null,
    "pickUpStore": null, //sucursal de retiro
    "payments": [ //pago
        {
            "date": "2025-10-07T12:41:52",
            "amount": 100,
            "couponAmount": 0,
            "status": "Approved", //estado del pago
            "method": "creditCard", //método del pago
            "integration": null, //puede ser el id de la pasarela de pagos
segun el canal
            "transactionFee": 0,
            "installments": 0, //cuotas
            "card": {
                    "paymentNetwork": "Visa",
                    "firstSixDigits": 0,
                    "lastFourDigits": 0,
                    "cardholderIdentificationNumber": null,
                    "cardholderIdentificationType": null,
                    "cardholderName": null
```

```
                    },
            "notes": null,
            "authorizationCode": "1565dad565a", //puede ser el código del
pago segun el canal
            "hasCancelableStatus": false,
            "id": 139474849
        }
    ],
    "shipments": [
        {
            "date": "2024-06-21T00:00:00", //fecha de creación del envío
            "products": [...],
            "method": {
                "trackingNumber": "nroSeguimiento", //numero de seguimiento
                "trackingUrl": "urlDeSeguimiento", //url del envio
                "courier": "FedEx", //transportadora
                "mode": null,
                "cost": 0,
                "type": "Ship",
                "eta": 0,
                "status": "InTransit" //estado del envio
            },
            "id": 110951151 //id del envio
        }
    ],
    "amount": 100, //monto total de la compra
    "shippingCost": 0, //costo de envio
    "financialCost": 0, //costo financiero
    "paidApproved": 100, //suma de los montos de los pagos aprobados
    "paymentStatus": "Approved", //estado del pago global
    "deliveryStatus": "InTransit", //estado del envio global
    "paymentFulfillmentStatus": "Done",
    "deliveryFulfillmentStatus": "Pending",
    "deliveryMethod": "Ship", //forma de entrega
    "paymentTerm": "Advance", //condición de pago
    "currency": "Local",
    "isOpen": true, //si esta abierta
    "isCanceled": false, //si esta cancelada
    "cartId": null,
    "draft": false,
    "promiseDeliveryDate": null,
    "promiseDispatchDate": null,
    "hasAnyShipments": true, //si tiene envios
    "hasAnyPayments": true, //si tiene pagos
    "date": "2025-10-07T12:41:52", //fecha de ingreso a Producteca
    "id": 170661260 //id de la venta en Producteca
}
```

Anexo 2: Código de `"salesChannel"` por canal

Agrupate (401)
Addi (549)
Amazon (90)
AMEX / Foodies Store / TBBM (338)
Banco Ciudad (294, 323)
Banco Colombia (329)
Banco Galicia (276)
Banco Macro (490, 537)
Banco Patagonia (341)
Banco Provincia (382, 572)
Bancolombia (329)
Bancor (396)
BBVA (140)
BNA (259, 268, 291, 298, 386)
Buybuy (400)
Carrefour (381)
Cetrogar (332)
Ceven (86, 331)
Claroshop (208)
Coppel (246)
Cornershop (261)
Cronishop (408)
Cyberpuerta (325)
Dafiti Chile (502)
Dafiti Colombia (286)
Diners (344)
Directv (395)
Doto (392)
Elektra (529)
Falabella Chile (330)
Falabella Colombia (383)
Falabella Peru (351)
Fenicio (347)
Foodies Store (AMEX) (338)
Foschia (556)
Fravega (220)
Galicia (276)
Garbarino (405)
Global Reward Solutions (411)
Hites (550)
ICBC (59)
Infobae (487)
Jumbo Colombia (447)
Juntoz (394)
Kodear (504)
La Marina (402)
Liverpool (262)
Magento (51)
Megatone (269)
MercadoLibre (2, 272)
Necxus (335)
On City (Ex Me Gusta) (288)
Paris (250)
Prestashop (391)
Quickfit (375)
Rappi (389)
Ripley Chile (237)
Ripley Perú (297)
Sam's-DSV (273)
Sanborns (536)
Sears (528)
Shein (314)
Shell (574)
Shopify (60, 253, 263, 299)
Sporting (583)
Supervielle (141)
Surtidora Departamental (452)
Tata (274)
Tecnofan (364)
Tienda Clic (69, 588)
Tienda Columbia (296)
Tienda Indigo (789)
Tienda Itau (275)
Tienda Nube (345, 580)
Total Play (492)
Uber (397)
Vtex (33, 398, 508)
Walmart Mexico (209, 320, 321)
Walmart Chile (499)
Walmart-DSV (233)
WooCommerce (393, 439, 442, 457)
Yape (581)